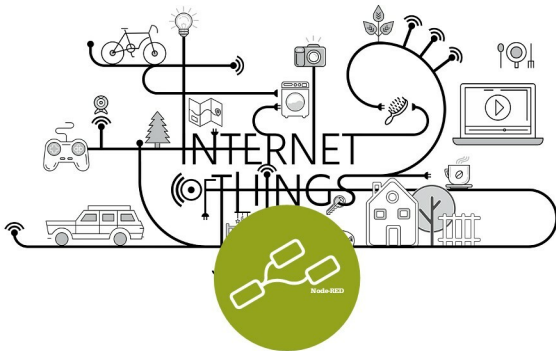


Writing Custom Add-ons for Node-RED

If you are planning to kickstart your IoT development plans but aren't comfortable with any particular programming environment, Node-RED could prove to be just right for you. You can use it to start developing applications with zero or minimal programming effort, and focus more on actual computing, interfacing and communication.



Node-RED is a visual wiring tool initially designed for IoT needs, but is more of a general-purpose tool these days. It can be used for interfacing sensor data, wired/wireless communication, cloud connectivity, data base connectivity, social media alerts, data analysis, system monitoring and, of course, any Node.js functionality. IBM Bluemix, SenseTecnica FRED, AT&T Flow and RedConnect.io are some of the options for getting an instance on the cloud with customised nodes and enterprise grade support. Node-RED forms a part of the Intel IoT gateways as a bridging solution. And it also powers IBM Watson Services.

Installation and initial steps

Node-RED can be installed on top of any Node.js runtime using the npm package manager, but certain nodes are platform-specific (say, with dependencies in Linux), and some are precisely targeted at boards like RaspberryPi for physical computing purposes. More nodes can also be added using npm. Node-RED can be easily managed using *pm2*, a process manager for Node.js for starting and stopping the instance. It can even run on Android

using Termux support, and talk to Arduino-type of microcontrollers through the Firmata protocol. Node-RED can be deployed on any cloud environment with Node.js runtime. Node-RED can also be run in a Docker container, as documented in nodered.org/docs/platforms/docker. It pulls the image the first time and runs in a container. You can add all necessary add-ons to make a custom bundle, as mentioned in the *Customising* section.

Once Node-RED has started, follow the *Getting Started* section of the documentation and create simple flows using the UI editor available on Port No. 1880 of the host. Data flows across the nodes through input, output connectors in the form of JavaScript objects named *msg*, with *msg.payload* as the key property; function nodes can be used to provide custom logic in between with minimal JavaScript coding.

Here are some nodes of interest for prototyping IoT solutions:

- MQTT out, MQTT in for publish and subscribe.
- Http-request for invoking REST APIs.
- JSON node for stringification or parsing.
- Trigger, delay nodes to control the message flow.